
ABSTRACT

Diagnosing malignant lymphoma, a complex cancer affecting the lymphatic system, presents significant challenges due to its diverse subtypes, including chronic lymphocytic leukemia (CLL), follicular lymphoma (FL), diffuse large B-cell lymphoma (DLBCL), and mantle cell lymphoma (MCL). This project introduces a deep learning framework that integrates both ensemble and non-ensemble transfer learning techniques to enhance diagnostic accuracy. By leveraging advanced feature extraction and classification strategies, the framework addresses the intricacies of lymphoma subtype identification, demonstrating the potential of deep learning in improving diagnostic precision and reliability. The proposed framework effectively classifies malignant lymphoma subtypes with strong performance metrics. The ensemble model, combining InceptionV3, Xception, and a custom CNN, achieves 96% training accuracy and 97% validation accuracy, with training and validation losses converging to approximately 0.1. Non-ensemble models also show competitive performance, with one configuration reaching 91% training accuracy and 89.5% validation accuracy, while another attains 85% training accuracy, closely aligning with validation performance. These results indicate the model's ability to generalize effectively, demonstrating minimal overfitting and stable learning across epochs. The framework offers a scalable solution for lymphoma subtyping, with robust diagnostic accuracy. While minor challenges remain due to overlapping features among subtypes, the findings emphasize the potential of deep learning in advancing lymphoma diagnosis and improving clinical decision-making.

CONTENTS

	ABSTRACT	
	LIST OF FIGURES	
1	INTRODUCTION.....	8
	1.1 Motivation.....	10
	1.2 OBJECTIVE.....	10
	1.3 SCOPE.....	10
2	LITERATURE REVIEW	11
	2.1 ML Based Approaches for Malignant Lymphoma Classification.....	11
	2.2 DL Based Approaches for Malignant Lymphoma Classification.....	11
	2.3 TL Based Approaches for Malignant Lymphoma Classification.....	12
3	METHODOLOGY.....	13
	3.1 Data Collection.....	13
	3.2 Pre-Processing on Histopathological Images.....	14
	3.3 Pre-Processing on Lymphoma Patch Images	15
	3.4 Data Augmentation Using GANs.....	17
	3.5 Feature Extraction and Classification	18
	3.6 Feature Explainability Using LIME.....	19
	3.7 Evaluation Metrics For Proposed Models	19
4	RESULTS.....	20
5	CONCLUSION.....	27
	REFERENCES.....	28
	APPENDIX.....	30

LIST OF FIGURES

No.	Title	Page No.
1.1	The Lymphatic System	9
3.1	Sample images of FL, CLL, MCL	13
3.2	Pre-processing on Histopathological Image	14
3.3	Image Pre-processing	15
3.4	Architecture of GANs	16
4.1	Confusion Matrix of Classification Results Using VGG16	20
4.2	VGG16 Training and Validation Accuracy Curve	21
4.3	VGG16 Training and Validation Loss Curve	21
4.4	Confusion Matrix of Classification Results Using InceptionV3	22
4.5	InceptionV3 Training and Validation Accuracy Curve	22
4.6	InceptionV3 Training and Validation Loss Curve	23
4.7	Confusion Matrix of Classification Results Using Xception	23
4.8	Xception Training and Validation Accuracy Curve	24
4.9	Xception Training and Validation Loss Curve	24
4.10	Confusion Matrix of Classification Results Using Ensemble Model	25
4.11	Ensemble Model Training and Validation Accuracy Curve	26
4.12	Ensemble Model Training and Validation Loss Curve	26

CHAPTER 1

INTRODUCTION

Lymphoma is a type of cancer that affects the lymphatic system a crucial part of the immune system responsible for fighting infections and maintaining fluid balance in the body. The lymphatic system figure 1.1 includes lymph nodes, the spleen, thymus gland, and bone marrow. Lymphoma occurs when lymphocytes, a type of white blood cell, grow uncontrollably, forming tumors in the lymph nodes or other parts of the body. There are two main categories of lymphoma: Hodgkin lymphoma (HL) and non-Hodgkin lymphoma (NHL), with the latter being more prevalent. Each type has distinct characteristics, treatment protocols, and prognoses. Types of Lymphoma includes Hodgkin and Non-Hodgkin lymphoma. Hodgkin lymphoma (HL) is characterized by the presence of Reed-Sternberg cells, large abnormal lymphocytes. It typically follows a predictable pattern of spread, beginning in the lymph nodes and moving to other parts of the lymphatic system. Non-Hodgkin lymphoma (NHL), on the other hand, encompasses a diverse group of lymphomas that differ in their growth rates, cell origins, and responses to treatment. NHL is more common and includes various subtypes, with B-cell lymphoma being the most prevalent.

B-cell lymphoma is a subset of non-Hodgkin lymphoma that originates from B-lymphocytes, cells that are integral to the immune response by producing antibodies. B-cell lymphomas account for the majority of NHL cases and can be categorized based on their aggressiveness and the specific nature of the cancer cells. Understanding the subtype of B-cell lymphoma is essential for determining the appropriate treatment approach and predicting patient outcomes. Several types of B-cell lymphoma exist, each with unique clinical features and treatment responses. Diffuse Large B-Cell Lymphoma (DLBCL) is the most common and is known for its rapid growth and potential to affect multiple organs. Follicular Lymphoma (FL) is a slower-growing type that may present with painless lymph node swelling. Chronic Lymphocytic Leukemia/Small Lymphocytic Lymphoma (CLL/SLL) involves either the blood and bone marrow or the lymph nodes, respectively. Mantle Cell Lymphoma (MCL) is a rare, aggressive type often involving the gastrointestinal tract.

The diagnosis of lymphoma involves a thorough imaging tests such as CT and PET scans, and a biopsy of affected tissues. The choice of treatment depends on the lymphoma's type and stage and may include chemotherapy, radiation therapy, immunotherapy, targeted therapy, or stem cell transplantation. Advances in medical research have led to more effective targeted therapies, improving survival rates and quality of life for patients with B-cell lymphoma. Understanding the specific characteristics of the lymphoma subtype is crucial for tailoring treatment and improving patient outcomes.

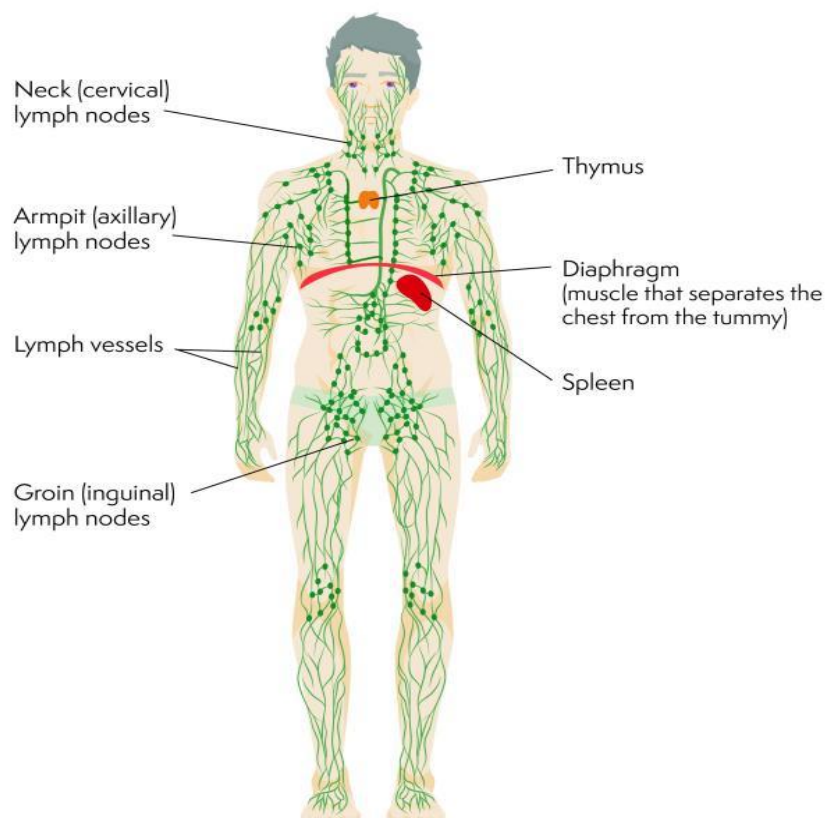


Fig 1.1 The Lymphatic System

1.1 MOTIVATION

Imagine being told you have cancer, but your doctors can't immediately pinpoint what kind or how fast it might spread. For many facing lymphoma, this uncertainty is a harsh reality. Lymphoma isn't just one disease it's a complex group of cancers that hijack the body's immune defenses, leaving patients and families scrambling for answers. While treatments exist, their success hinges on knowing exactly which subtype of lymphoma someone has. A misdiagnosis or delayed classification could mean life or death, especially with aggressive forms like diffuse large B-cell lymphoma (DLBCL), which can overwhelm the body in weeks if untreated. For others, like those with rare mantle cell lymphoma, limited awareness among doctors leads to delayed care. Their stories drive this project. By untangling the nuances of B-cell lymphomas their triggers, behaviors, and responses to therapies. The aims to replace fear with clarity, guesswork with precision, and despair with hope. Because no one should fight cancer in the dark.

1.2 OBJECTIVE

The primary objective of the project is to develop an advanced deep learning framework for the accurate classification of B-cell Non-Hodgkin Lymphomas (NHL), specifically focusing on subtypes such as Follicular Lymphoma (FL), Chronic Lymphocytic Leukemia (CLL), Mantle Cell Lymphoma (MCL), and Diffuse Large B-Cell Lymphoma (DLBCL). Given the complexity of lymphoma diagnosis and the limitations of traditional histopathological assessments, this research aims to leverage transfer learning and Generative Adversarial Networks (GANs) to enhance the classification accuracy. By improving automated lymphoma classification, this study seeks to provide a reliable and efficient diagnostic tool to assist pathologists, reduce diagnostic errors, and facilitate early and precise treatment planning.

1.3 SCOPE

This analysis serves as a comprehensive resource for clinicians to refine diagnostic workflows and tailor treatment plans. Researchers benefit from synthesized data on genetic alterations and therapeutic targets, aiding drug development. Policymakers can leverage epidemiological insights to prioritize healthcare funding, while patients gain clarity on prognosis and care options. Educational institutions may use this framework for training oncologists, fostering interdisciplinary collaboration to advance lymphoma management globally.

CHAPTER 2

LITERATURE REVIEW

2.1 ML Based Approaches for Malignant Lymphoma Classification

Victor Bobée et al. [8] developed a gene expression assay, combining ligation-dependent RT-PCR and next-generation sequencing to evaluate over 130 genetic markers. This assay captures key gene expression signatures of B-NHL cells and their microenvironment. A random forest algorithm, trained on over 400 annotated cases, facilitates accurate classification. It effectively reduces misclassification in low-grade lymphomas and identifies critical features in high-grade lymphomas, such as cell-of-origin signatures and MYC and BCL2 expression levels. This assay enhances diagnostic precision and supports patient management and clinical trial stratification. Zhang et al. [9] explored gene expression profiling using 1039 samples from 27 GEO datasets to identify highly and differentially expressed genes among SBCLNs. A classifier, based on the NanoString platform and trained on samples from 57 SBCLN cases and 102 nonmalignant controls, utilized a cascade binary classification method and a random forest algorithm with 35 gene signatures. The classifier was validated on an independent cohort of 197 SBCLN patients, achieving >95% sensitivity and specificity for cases with tumor cell content above 0.72. Incorporating genetic aberrations such as IGH-BCL2 translocation, MYD88 L265P, and BRAF V600E mutations, the model reached optimal sensitivity and specificity of 0.88 and 0.98, respectively.

2.2 DL Based Approaches for Malignant Lymphoma Classification

Zeyad Ghaleb Al-Mekhlafi et al. [10] proposed two systems for classifying malignant lymphomas using two datasets. The first system combined DenseNet-121 and ResNet-50 for deep feature extraction, reduced dimensionality using principal component analysis (PCA), and classified the features with a support vector machine (SVM). The second system combined features from DenseNet-121, ResNet-50, and hand-crafted algorithms like gray-level co-occurrence matrix (GLCM) and local binary pattern (LBP), classifying them with a feed-forward neural network (FFNN). Chava Perry et al. [11] introduced a novel deep learning tool for diagnosing DHLs/THLs directly from scanned biopsy slide images. The study included 57 biopsies, with 32 in a training set (five DH lymphoma cases) and 25 in a validation set (ten DH/TH cases).

2.3 TL Based Approaches for Malignant Lymphoma Classification

Transfer learning (TL) leverages pre-trained models to expedite Lymphoma classification from patch image, offering faster and more efficient performance compared to traditional ML and DL approaches. TL reduces the need for large labelled datasets and extensive processing resources. Schahrazad Soltane et al. [12] developed a deep learning-based method for lymphoma classification. They utilized two datasets to evaluate their approach. The proposed method employed ResNet50 for feature extraction, followed by principal component analysis (PCA) for dimensionality reduction and a support vector machine (SVM) for classification. Alternatively, they implemented a second approach using a combination of DenseNet121, local binary patterns (LBP), and convolutional neural network (CNN). Jianfei Zhang et al. [13] proposed a method for classifying non-Hodgkin's lymphoma (NHL) subtypes using a fusion of transfer learning (TL) and principal component analysis (PCA). The digital pathology images were preprocessed through segmentation and division, followed by fine-tuning pre-trained models for feature extraction. PCA was applied to map the extracted features, which were then classified using a neural network.

CHAPTER 3

METHODOLOGY

3.1 DATA COLLECTION

The datasets utilized in this project were meticulously sourced from publicly available repositories to ensure high-quality and high-resolution histopathological image data suitable for advanced computational analysis and machine learning applications. This approach allowed access to comprehensive datasets representing various lymphoma subtypes, thereby enabling a robust classification and predictive modeling framework.

For Follicular Lymphoma (FL), Chronic Lymphocytic Leukemia (CLL), and Mantle Cell Lymphoma (MCL), datasets were obtained from the Multi Cancer Dataset on Kaggle, a decentralized platform providing an extensive collection of publicly shared academic datasets. These datasets are available in the Tagged Image File Format (.tif), a high-resolution image format widely regarded for its capability to preserve intricate histopathological details. The use of .tif images ensures compatibility with diverse image preprocessing techniques, including normalization, augmentation, and feature extraction. The rich resolution and clarity of these images allow researchers to analyze subtle morphological variations critical for differentiating between the aforementioned lymphoma subtypes.

For Diffuse Large B-cell Lymphoma (DLBCL), data were acquired from the National Cancer Institute's (NCI) Cancer Imaging Program. This dataset is specifically tailored to provide high-quality histopathological images with detailed tissue structure information. The histopathological format is pivotal for analyzing tissue morphology and cellular characteristics that are instrumental in classifying DLBCL with precision. The dataset from the NCI Cancer Imaging Program includes annotations and is curated to support both visual and computational examination of lymphoma histology. The detailed tissue representations are critical for tasks such as feature extraction, texture analysis, and the development of explainable AI models.

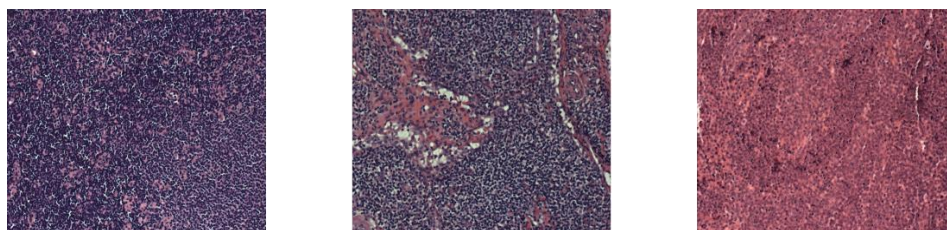


Fig 3.1 Sample images of FL, CLL, MCL.

3.2 PRE-PROCESSING ON HISTOPATHOLOGICAL IMAGE

Preprocessing histopathological images of Diffuse Large B-Cell Lymphoma (DLBCL) is crucial for ensuring accurate analysis and diagnosis. These images, derived from microscopic examination of stained lymphoma tissue, often exhibit challenges like variable staining, noise, and the presence of non-tissue regions. A robust preprocessing pipeline addresses these issues while preserving the structural details essential for identifying lymphoma-specific features. Resizing is a vital step to standardize image dimensions for tasks such as training deep learning models. Given the high resolution of histopathological images, resizing reduces computational overhead and ensures dataset uniformity. It is critical to maintain the aspect ratio during resizing to preserve DLBCL-specific tissue structures, preventing distortions that might compromise analysis. Common interpolation methods like bilinear or bicubic are employed to achieve a balance between smoothness and clarity.

Background removal is another essential step in preprocessing DLBCL histopathological images, focusing on isolating lymphoma tissues from non-tissue regions like slide backgrounds or preparation artifacts. Such extraneous elements introduce noise that can skew analysis if not addressed. Thresholding techniques are often applied to distinguish tissue from the background based on pixel intensities. Adaptive thresholding provides more precision by dynamically adjusting thresholds to local intensity variations, while morphological operations like erosion and dilation refine tissue boundaries. In cases of complex artifacts or irregular tissue borders, deep learning models trained on annotated DLBCL datasets deliver high-accuracy segmentation, ensuring the focus remains on diagnostically relevant tissue regions.

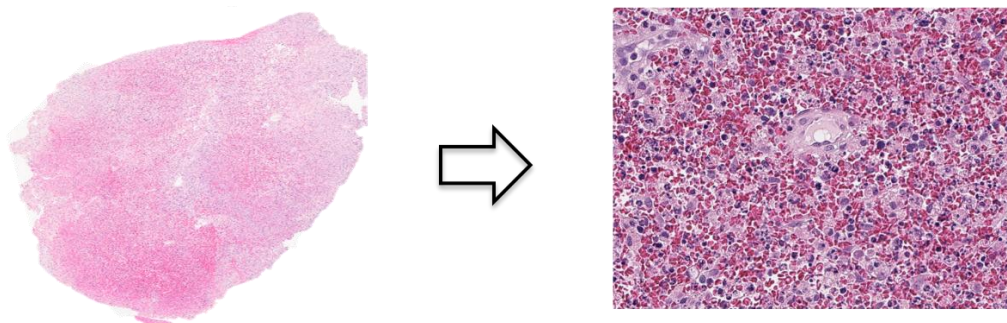


Fig 3.2 Pre-processing on Histopathological Image

3.3 PRE-PROCESSING ON LYMPHOMA PATCH IMAGES

Each patch image in the lymphoma dataset undergoes a series of essential preprocessing steps aimed at enhancing interpretability, reducing distortions, and improving image quality. These steps are crucial for ensuring that the images are consistent, uniform, and of high quality, which is necessary for accurate analysis and classification tasks. Medical imaging, particularly in the context of lymphoma, often presents challenges like uneven lighting due to equipment degradation or defective imaging conditions. Such lighting inconsistencies can obscure important details, making it difficult to interpret the images. Additionally, variability in image intensity levels, often due to differing acquisition settings, can further complicate the analysis.

3.3.1 GAUSSIAN FILTERING

Gaussian filtering is a key image preprocessing technique that plays an important role in smoothing and noise reduction while preserving critical image features. In the context of lymphoma patch images, applying Gaussian filtering helps ensure that the dataset is clean and well-suited for further analysis. The process involves using a Gaussian kernel, which is a square matrix of values derived from the Gaussian function. The filter is applied by sliding the kernel across the image, computing a weighted sum of the surrounding pixel values. The weights are determined by the Gaussian distribution, with higher weights assigned to pixels closer to the centre of the kernel and lower weights given to those farther away. This means that each pixel in the image is replaced by a weighted average of its neighbouring pixels, with nearby pixels having more influence on the result.

The use of a Gaussian filter helps smooth out rapid intensity variations and removes high-frequency noise, which is common in medical imaging due to acquisition artifacts or other imaging conditions. This smoothing process reduces visual distortions while preserving key features such as edges and structures, which are essential for accurately analysing the lymphoma patches. Gaussian filtering also ensures that the images maintain a consistent quality, making it easier for machine learning models to extract meaningful patterns and make predictions. The kernel size, in this case, is set to 5×5 pixels. This size is a good balance between reducing noise and retaining sufficient detail for the model to analyse. The standard deviation (σ) of the Gaussian distribution controls the extent of the smoothing. A moderate σ ensures that the filter is effective in reducing noise without overly blurring important structures in the image.

3.3.2 IMAGE RESIZING

Resizing plays a crucial role in optimizing neural network performance by enabling the model to capture relevant features and patterns effectively. It also addresses computational memory limitations, reduces overfitting, and facilitates transfer learning. In this dataset, lymphoma patch images were resized from their original dimensions of 1388×1040 pixels to 224×224 pixels. This resizing ensures the neural network processes data more efficiently, leading to improved accuracy and computational performance. The adjustment of image dimensions enhances inference speed and augments the dataset for better model generalization.

3.3.3 IMAGE SCALING

In the preprocessing pipeline, pixel values are normalized to improve model efficiency. Original 8-bit pixel intensity values ranging from 0 to 255 are scaled down to a range of 0–1 by dividing by 255. This normalization not only speeds up neural network training but also ensures consistent gradient behaviour during backpropagation. The brightness and contrast adjustments are simplified, with brightness set to 1 and contrast to 0, ensuring that visual consistency is maintained while optimizing computational efficiency. By applying this scaling approach, the pixel values are standardized, leading to improved model training and performance.

3.3.4 IMAGE ZOOMING

To further enhance robustness, a random zoom transformation is applied, with a zoom range allowing adjustments of up to 20%. This introduces variability in scale and perspective, helping the model learn to handle images with different dimensions and viewpoints of the same class. Following zooming, horizontal flipping is applied randomly, mirroring the images along the horizontal axis.

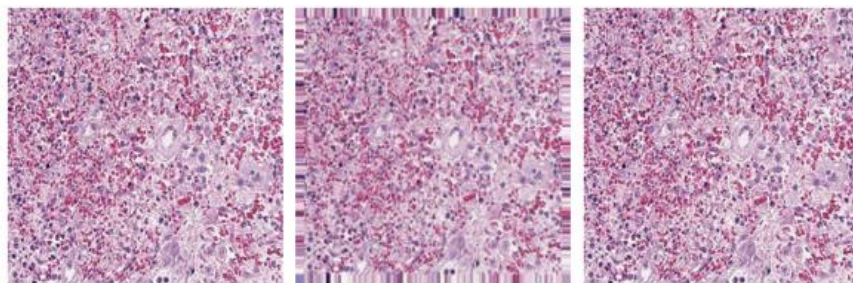


Fig 3.3 Image Pre-processing

3.4 DATA AUGMENTATION USING GANs

Generative Adversarial Networks (GANs) are crucial in augmenting datasets, particularly when dealing with limited data availability, such as Lymphoma patch images. GANs enhance neural network performance by generating synthetic images, thus increasing the diversity of the training set and reducing the risk of overfitting. Applying transformations like rotations, flips, and crops, along with synthetic image generation, makes the model more robust and capable of generalizing to new, unseen data.

In this approach, GANs were trained using approximately 3000 real Lymphoma patch images, subsequently generating around 2000 synthetic images per class. This expansion significantly boosts the dataset size, addressing the imbalance between positive and negative cases of B-cell non-Hodgkin lymphoma. The augmented dataset enables the model to learn from a more comprehensive range of examples, improving its accuracy and robustness in classification tasks.

Furthermore, by generating synthetic images that closely resemble test data, GANs help in enhancing the model's performance on the test set. This strategy ensures that the model can handle variations in real-world clinical data. The effectiveness of this data augmentation technique is validated through kernel density estimation, comparing the distribution of real and generated images, ensuring that the synthetic data aligns well with the actual data.

By integrating GANs into the training process, models benefit from a richer, more diverse dataset, leading to improved generalization and performance. This method is particularly valuable in medical image analysis, where data scarcity and class imbalance are common challenges. Through the generation of realistic synthetic samples, GANs facilitate the development of more accurate and efficient models for classifying B-cell non-Hodgkin lymphoma, ultimately enhancing clinical decision-making and patient outcomes.

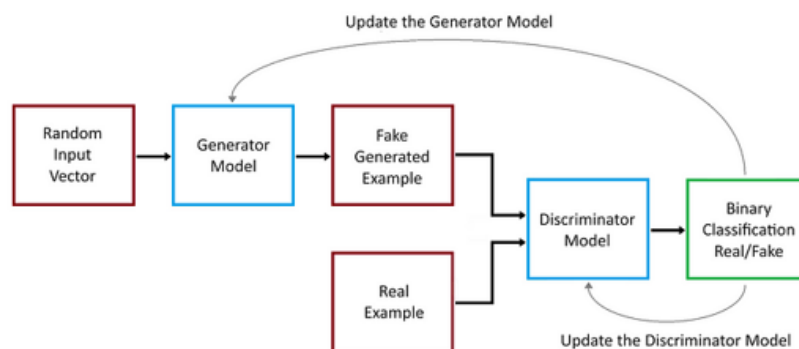


Fig 3.4 Architecture of GANs

3.5 FEATURE EXTRACTION AND CLASSIFICATION

Transfer Learning (TL) is a powerful approach that enhances model performance on specific tasks by leveraging pre-trained models. Key TL techniques include fine-tuning, which adapts a pre-trained model for a new task, and feature extraction, which utilizes the learned features from an existing model to train a new classifier. Domain adaptation reduces discrepancies between source and target domains, while model pre-training involves initializing a model with weights from a pre-trained model. Multi-task learning enables models to handle multiple tasks concurrently. These methods are especially useful when labelled data is limited, improving model performance and accelerating training.

For the classification of B-cell non-Hodgkin lymphoma using patch images, TL was applied using several advanced models, including VGG-16, Inceptionv3, and Xception. These models were adapted by removing their top classification layers, retaining their convolutional structures to extract significant visual features from the patch images. The images were pre-processed to meet the input specifications of these models, and features were extracted from deeper layers that captured hierarchical visual representations. The extracted features provided a detailed depiction of the visual characteristics of the patch images, which were used for the classification task. The choice of layer for feature extraction was critical as it influenced the level of abstraction captured in the embeddings.

An ensemble model combining Inceptionv3, Xception, and a custom CNN was also employed to enhance classification accuracy. This ensemble approach leveraged the complementary strengths of the different architectures, resulting in a more robust and accurate classification process. By integrating these models, the ensemble model could capture a wider range of features, improving the overall performance in classifying B-cell non-Hodgkin lymphoma. This approach demonstrated the effectiveness of TL and ensemble learning in medical image classification, providing a robust solution to the challenges posed by limited data availability and the complexity of medical image analysis.

3.6 FEATURE EXPLAINABILITY USING LIME

LIME is a method designed to make complex machine learning models more interpretable at a local level, focusing on explaining individual predictions. It generates perturbed samples by introducing random variations in feature values around a specific data instance, allowing it to approximate the model's decision boundary for that case. LIME builds a locally interpretable model, usually a linear one, to explain how the original model made its decision. By using kernelized weights, LIME ensures that samples closer to the original instance have more influence on the explanation. It analyzes feature importance to determine which features had the greatest impact on the prediction. This is particularly valuable in fields like healthcare and finance, where understanding the reasoning behind individual predictions is critical. LIME enhances transparency and trust in machine learning models, making them easier to deploy and interpret in real-world applications.

3.7 EVALUATION METRICS FOR PROPOSED MODELS

Accuracy, precision, recall, and F1 score are performance metrics commonly used in classification problems to evaluate the performance of a model. The measure of accuracy is derived from the ratio of correctly classified points to the total number of points.

Accuracy: Accuracy measures the overall effectiveness of the model by calculating how often the model's predictions match the actual outcomes.

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

Precision: Precision evaluates how well the model minimizes false positives, indicating its reliability in correctly predicting tumor cases.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall: Also known as sensitivity, recall measures the model's ability to correctly detect all tumor instances.

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score: The F1-Score is the harmonic mean of precision and recall, providing a balanced measure that is particularly useful when dealing with imbalanced classes.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

CHAPTER 4

RESULTS

4.1 Non-ENSEMBLE MODELS

Non-ensemble models are individual machine learning models that make predictions without combining multiple models. These models include algorithms like decision trees, support vector machines, and neural networks, which learn patterns from data independently. Unlike ensemble models, they do not aggregate results from multiple models to improve performance. While simpler and faster to train, non-ensemble models may be more prone to overfitting or lower generalization compared to ensemble approaches.

4.1.1 PERFORMANCE ON VGG16

The confusion matrix (Figure 4.1) demonstrates the classification performance of the VGG16 model across the four classes. For Class 0 (CLL), 956 samples are correctly classified, with a few misclassified as Class 3 (MCL). Class 1 (DLBCL) achieves perfect classification, with all 860 samples accurately identified. Class 2 (FL) has 829 correct predictions but shows some misclassifications into Class 3 (109 cases) and Class 0 (62 cases). Similarly, Class 3 (MCL) is correctly predicted for 889 samples, though some confusion occurs with Class 0 (79 cases) and Class 2 (32 cases). These misclassifications suggest overlapping features among certain classes, particularly between FL, MCL, and CLL. Overall, the matrix highlights strong performance with minor areas for improvement.

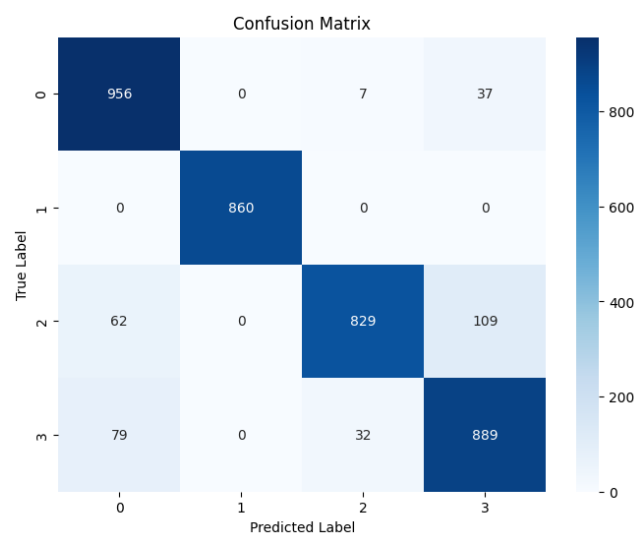


Fig 4.1 Confusion Matrix of Classification Results Using VGG16

The accuracy plot(Figure 4.2) illustrates a steady increase in training accuracy, surpassing 91% by the final epoch. Validation accuracy follows a similar trend with slight fluctuations, ultimately approaching the training accuracy. This consistency between training and validation accuracy suggests that the model generalizes well to unseen data and avoids significant overfitting.

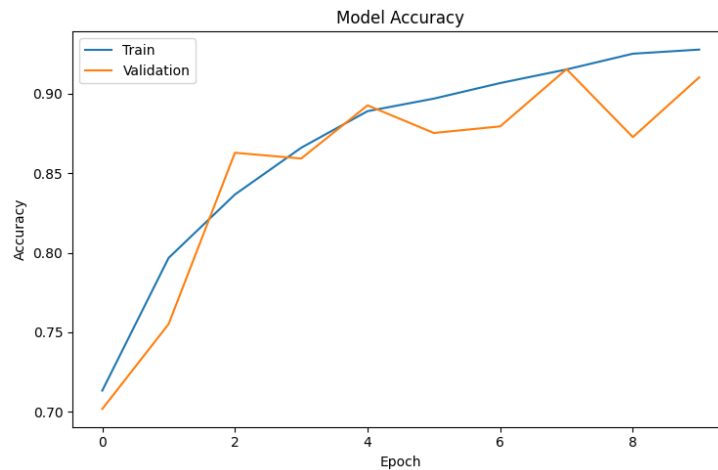


Fig 4.2 VGG16 Training and Validation Accuracy Curve

The loss curves(Figure 4.2) indicate a steady decrease in both training and validation loss over the epochs. The training loss reduces to approximately 0.220, while the validation loss shows a similar declining trend, albeit with minor oscillations. This behavior reflects the model's effective learning of features during training and its ability to minimize errors on both training and validation datasets.

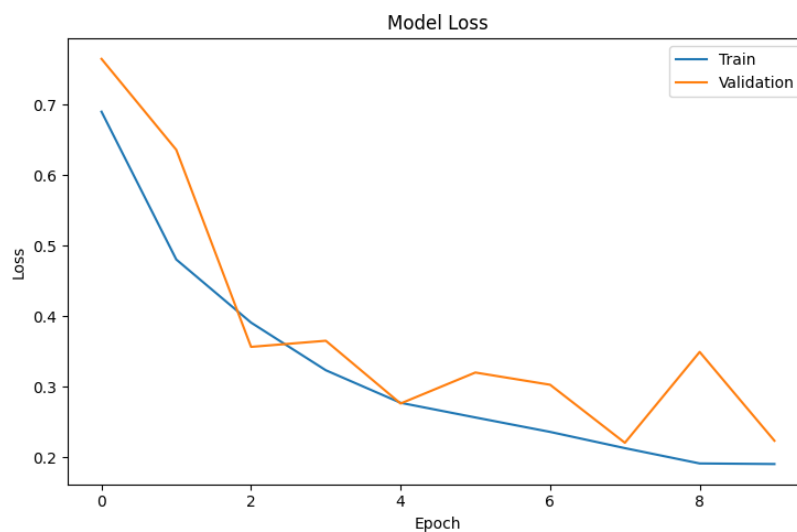


Fig 4.3 VGG16 Training and Validation Loss Curve

4.1.2 PERFORMANCE ON INCEPTIONV3

The confusion matrix (Figure 4.4) demonstrates the classification performance of the InceptionV3 model across the four classes. Most predictions are correct, with high values along the diagonal, indicating strong classification accuracy. There are some misclassifications across classes, particularly with slight confusion between certain classes. The overall performance suggests that the model achieves high accuracy, though some inter-class feature similarities may contribute to minor misclassifications.

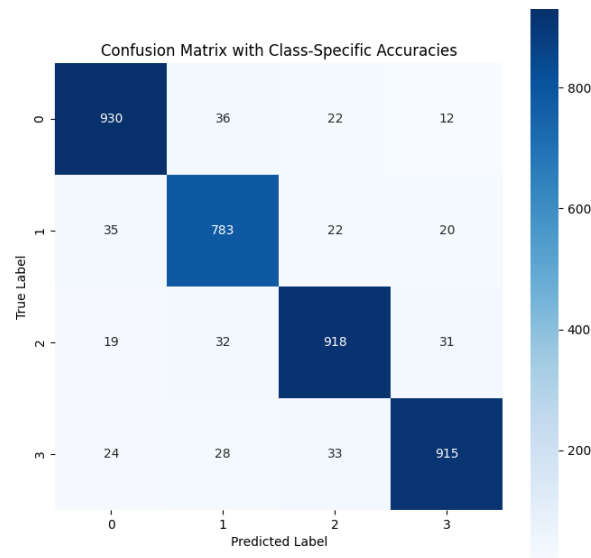


Fig 4.4 Confusion Matrix of Classification Results Using InceptionV3

The accuracy plot shows a steady increase in both training and validation accuracy over 30 epochs. The training accuracy surpasses 90%, while the validation accuracy follows a similar trajectory, albeit with minor fluctuations, reaching approximately 89.5%. The close alignment of training and validation accuracy suggests good generalization, with no significant signs of overfitting.

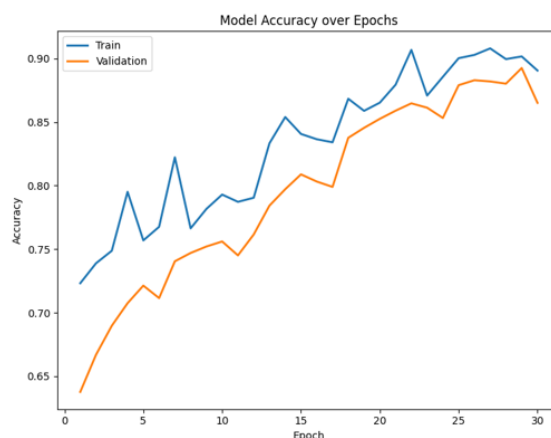


Fig 4.5 InceptionV3 Training and Validation Accuracy Curve

The loss plot illustrates a consistent decrease in both training and validation loss over epochs. The training loss drops to around 0.34, while validation loss follows a similar trend but with occasional fluctuations. These fluctuations indicate some variability in model performance on unseen data, but the overall trend confirms effective feature learning and error minimization.

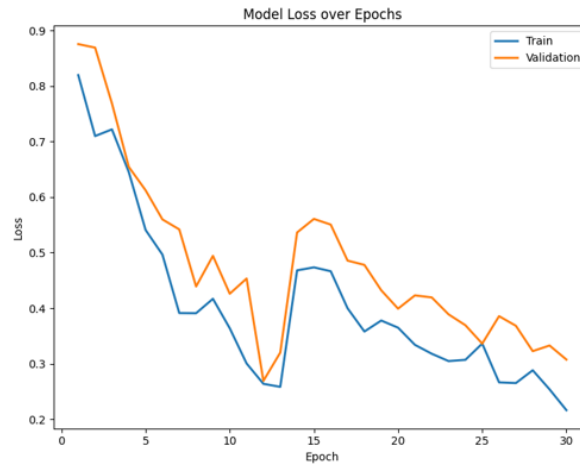


Fig 4.6 InceptionV3 Training and Validation Loss Curve

4.1.3 PERFORMANCE ON XCEPTION

The confusion matrix (Figure 4.7) reveals strong classification performance across the four classes, achieving an overall accuracy of 86.2%. While the majority of samples are correctly classified, there are notable areas of confusion between certain classes, particularly among those with overlapping features. These misclassifications highlight potential feature similarities that could be addressed to improve the model's precision.

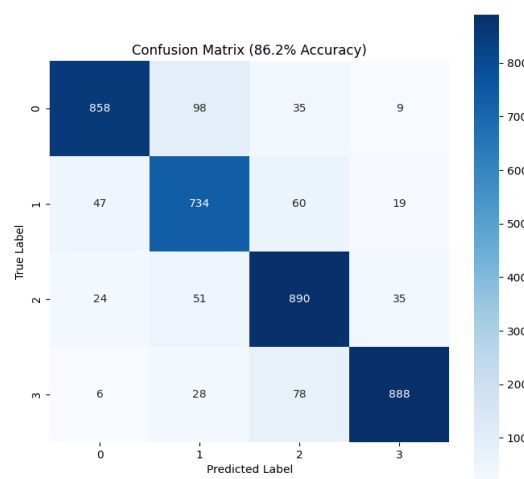


Fig 4.7 Confusion Matrix of Classification Results Using Xception

The accuracy plot (Figure 4.8) highlights a steady improvement in both training and validation accuracy over the epochs. Training accuracy reaches approximately 85%, with validation accuracy closely following, albeit with some fluctuations. This indicates that the model generalizes well to unseen data, maintaining a balance between fitting the training set and preserving validation performance.

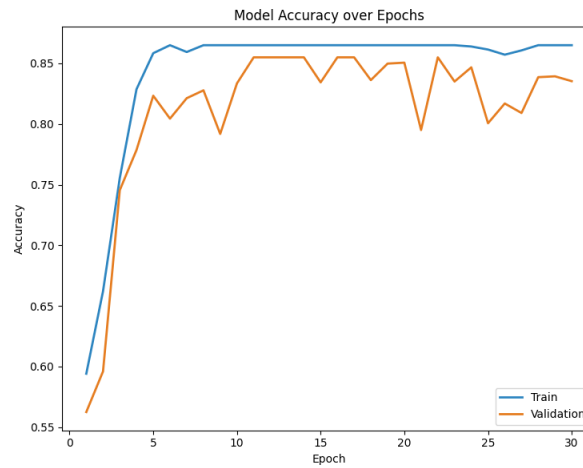


Fig 4.8 Xception Training and Validation Accuracy Curve

The loss curves (Figure 4.9) reflect a consistent reduction in training loss, stabilizing at around 0.25. Validation loss also decreases over time but exhibits more variability. Despite the oscillations, the overall trend suggests effective learning and a satisfactory reduction in error across both training and validation datasets.

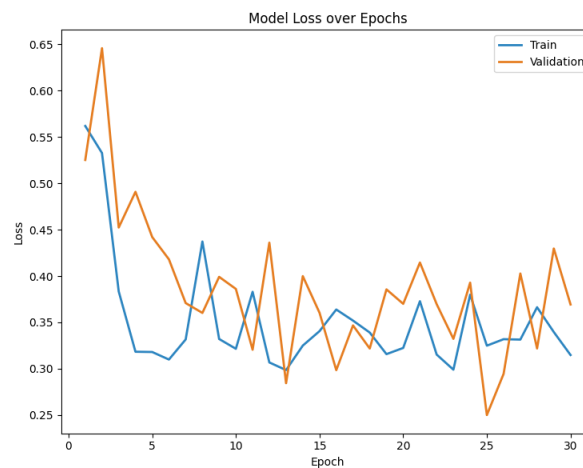


Fig 4.9 Xception Training and Validation Loss Curve

4.2 ENSEMBLE MODEL

The confusion matrix Figure 4.10 illustrates the classification performance of an ensemble model combining InceptionV3, Xception, and a custom CNN across four classes. For Class 0 (CLL), the ensemble model achieves 978 correct classifications, with minimal misclassifications observed into other classes, particularly Class 2 and Class 3. Class 1 (DLBCL) demonstrates strong performance with 841 correctly classified samples, though a few misclassifications occur into Classes 0 and 2. Similarly, Class 2 (FL) achieves 976 correct predictions, with minor misclassifications into Classes 0, 1, and 3. Class 3 (MCL) also performs exceptionally well, with 981 correct predictions, while some overlap in features leads to a small number of samples being classified as Classes 0, 1, and 2. These results highlight the ensemble model's robustness and accuracy, although slight misclassifications suggest the need for further optimization to distinguish overlapping features among certain classes.

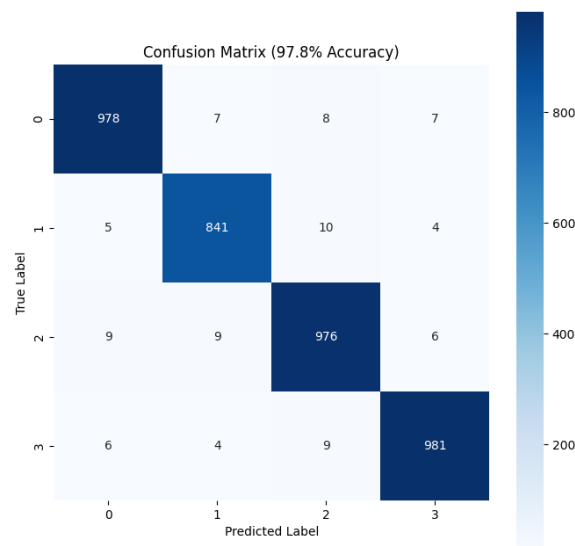


Fig 4.10 Confusion Matrix of Classification Results Using Ensemble Model

The accuracy plot Figure 4.11 shows a steady and consistent increase in training accuracy for the ensemble model, surpassing 96% by the final epoch. Validation accuracy follows a similar trend, peaking at approximately 97%, with only minor fluctuations throughout the training process. The close alignment between training and validation accuracies indicates that the ensemble model generalizes well to unseen data, effectively mitigating the risk of overfitting.

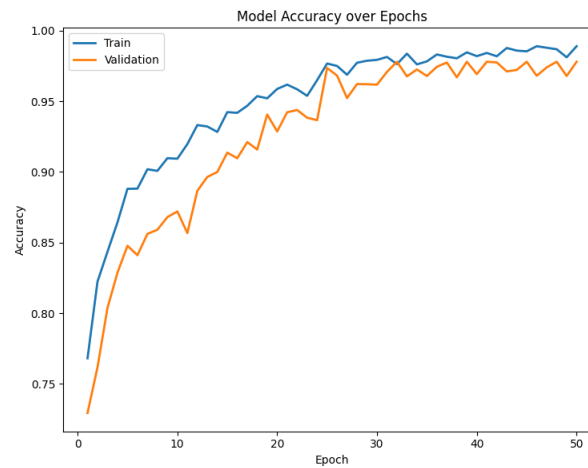


Fig 4.11 Ensemble Model Training and Validation Accuracy Curve

The loss curves Figure 4.12 reveal a gradual and consistent reduction in both training and validation loss over the course of training. The training loss decreases to approximately 0.1, while the validation loss mirrors this trend, exhibiting a steady decline with slight oscillations. This behaviour underscores the ensemble model's ability to learn features effectively and minimize classification errors. The combination of InceptionV3, Xception, and a custom CNN leverages the unique strengths of each architecture, contributing to the overall robustness of the model. Despite minor challenges with misclassifications, the ensemble model demonstrates strong generalization, effective learning, and high accuracy across all classes.

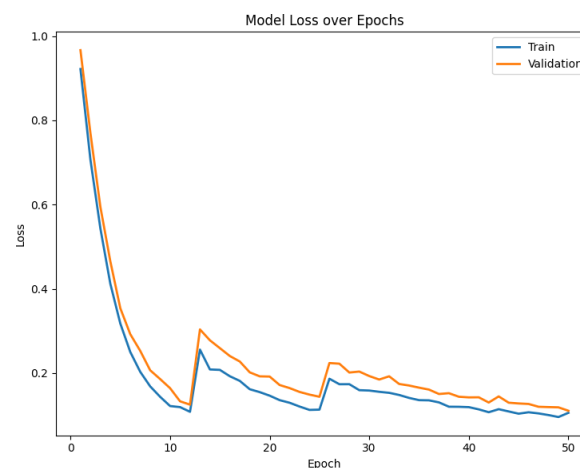


Fig 4.12 Ensemble Model Training and Validation Loss Curve

CHAPTER 5

CONCLUSION

The project proposes an innovative method for B-cell non hodgkin lymphoma classification, leveraging cutting-edge deep learning techniques such as transfer learning and Generative Adversarial Networks (GANs) to address challenges in Patch image analysis. The model enhances dataset diversity and robustness through pre-processing techniques and data augmentation with GANs, enabling better generalization across various patches. By employing transfer learning models such as VGG16, InceptionV3, Xception and ensemble model for feature extraction and classification, the approach significantly improves classification accuracy while minimizing overfitting. Furthermore, the incorporation of Explainable AI techniques, like LIME, ensures transparency by offering interpretable insights into feature importance, making the model both accurate and trustworthy for clinical applications. This approach provides medical professionals with a robust tool for timely and precise lymphoma diagnosis.

REFERENCES

- [1] Sivashankari Rajadurai, Kumaresan Perumal, Muhammad Fazal Ijaz, and Chiranji Lal Chowdhary. "PrecisionLymphoNet: Advancing Malignant Lymphoma Diagnosis via Ensemble Transfer Learning with CNNs", *Diagnostics*, 14(5), 469, Feb. 2024.
- [2] Fazal, I.M., Attique, M., & Son, Y. "Data-driven cervical cancer prediction model with outlier detection and over-sampling methods." *Sensors*, vol. 20, no. 2809, 2020.
- [3] Hasani, Paravastu, Farhadi, Yousefirizi, Morris, Rahmim, Roschewski, Summers, Saboury, "Artificial intelligence in lymphoma PET imaging: A scoping review (current trends and future directions)." *PET Clin.* 2022.
- [4] Zhao, M., Mallesh, N., Höllein, A., Schabath, R., Haferlach, C., Haferlach, T., Elsner, F., Lüling, H., Krawitz, P., Kern, W. "Hematologist-Level Classification of Mature B-Cell Neoplasm Using Deep Learning on Multiparameter Flow Cytometry Data," *Cytometry Part A*, 97(11), 1073–1080, 2020.
- [5] Sheng, B., Zhou, M., Hu, M., Li, Q., Sun, L., Wen, Y. "A blood cell dataset for lymphoma classification using faster R-CNN," *Biotechnology & Biotechnological Equipment*, 34(1), 413–420, 2020.
- [6] Lippi, M., Gianotti, S., Fama, A., Casali, M., Barbolini, E., Ferrari, A., Fioroni, F., Iori, M., Luminari, S., Menga, M., et al. "Texture analysis and multiple-instance learning for the classification of malignant lymphomas," *Computer Methods and Programs in Biomedicine*, 185, 105153, 2020.
- [7] Zhang, J., Cui, W., Guo, X., Wang, B., Wang, Z. "Classification of digital pathological images of non-Hodgkin's lymphoma subtypes based on the fusion of transfer learning and principal component analysis," *Medical Physics*, 47(9), 4241–4253, 2020.
- [8] Victor Bobée, Fanny Drieux, Vincent Sater, Liana Veresezan, & Philippe Ruminy, "A gene expression signature for the accurate diagnosis of B-cell non-Hodgkin lymphomas", *Blood Cancer Journal*, vol. 10, no. 9, pp. 1-12, May 2020.
- [9] Wei Zang, Qilin Ao, Yuqi Guan, Zhoujie Zho, Dong Kuang, Monica M.Q. Li,...& Min Xiao, "A novel diagnostic approach for the classification of small B-cell lymphoid neoplasms based on the NanoString platform", *Modern Pathology*, 35(5), 632–639, Nov. 2021.
- [10] Zeyad Ghaleb Mekhlafi, Ebrahim Senan, Meshari Alazmi,...& Mona Alshahrani, "Hybrid diagnostic method for malignant lymphoma classification using fusion features from histopathological images", *Electronics*, vol. 11, no. 18, pp. 2865, Sept. 2022.

- [11] Chava Perry, Orli Greenberg, Shira Haberman, Neta Herskovitz, Inbal Gazy & Irit Avivi, "Image-Based Deep Learning Detection of High-Grade B-Cell Lymphomas Directly from Hematoxylin and Eosin Images", *Cancers*, 15(21), Oct. 2023.
- [12] Schahrazad Soltana, Sameer Alsharif and Salwa M Serag Eldin, "Classification and Diagnosis of Lymphoma's Histopathological Images Using Transfer Learning", *Computer Systems Science and Engineering*, 40(2), 629–644, Sep. 2022.
- [13] Jianfei Zhang, Wensheng Cui, Xiaoyan Guo, and Zhen Wang, "Classification of digital pathological images of non-Hodgkin's lymphoma subtypes based on the fusion of transfer learning and principal component analysis," *Medical Physics*, vol. 47, no. 9, pp. 4241–4253, Sep. 2020.
- [14] Hamdi, M., Senan, E.M., Jadhav, M.E., Olayah, F., Awaji, B., Alalayah, K.M. "Hybrid Models Based on Fusion Features of a CNN and Handcrafted Features for Accurate Histopathological Image Analysis for Diagnosing Malignant Lymphomas," *Diagnostics*, 13(9), 2258, 2023.
- [15] Syrykh, C., Abreu, A., Amara, N., Siegfried, A., Maisongrosse, V., Frénois, F.X., Brousset, P. "Accurate diagnosis of lymphoma on whole-slide histopathology images using deep learning," *NPJ Digital Medicine*, 3(1), 63, 2020.
- [16] Hagar, M., Elsheref, F.K., Kamal, S.R. "A New Model for Blood Cancer Classification Based on Deep Learning Techniques," *International Journal of Advanced Computer Science and Applications*, 14(6), 422–429, 2023.

APPENDIX

SAMPLE CODE

```
import zipfile
import os
import openslide
import numpy as np
import cv2
from sklearn.model_selection import train_test_split
from tqdm import tqdm
from PIL import Image
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt
from tensorflow.keras.applications import DenseNet169
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.densenet import preprocess_input
from tensorflow.keras.utils import to_categorical
from sklearn.preprocessing import LabelEncoder

#unzipping data
zip_file = "/home/jupyter-jishnu_2325_sair/Project Phase 1/Data/4L.zip"

extract_dir = "/home/jupyter-jishnu_2325_sair/Project Phase 1/Data"
os.makedirs(extract_dir, exist_ok=True)

with zipfile.ZipFile(zip_file, 'r') as zip_ref:
    zip_ref.extractall(extract_dir)

print(f"Data extracted to: {os.path.abspath(extract_dir)}")

#preprocessing on histological image

def is_useful_patch(image, white_threshold=220, tissue_percentage=0.5):
    """
    Check if the patch contains enough tissue (non-white area).

    Args:
    image: numpy array of the image patch
    white_threshold: pixel values above this are considered white
    tissue_percentage: minimum required percentage of non-white pixels

    Returns:
    bool: True if the patch is useful (contains enough tissue), False otherwise
    """
    # Convert to grayscale if it's a color image
    if len(image.shape) == 3:
        gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
    else:
        gray = image
```

```
# Create a binary mask of non-white pixels
non_white_mask = gray < white_threshold

# Calculate the percentage of non-white pixels
non_white_percentage = np.sum(non_white_mask) / non_white_mask.size

return non_white_percentage > tissue_percentage

def extract_patches(slide_path, patch_size, num_patches=145, max_attempts=10000):
    """
    Extract patches from the slide, excluding mostly white patches.

    Args:
    slide_path: path to the .svs file
    patch_size: tuple of (width, height) for the patches
    num_patches: number of patches to extract
    max_attempts: maximum number of attempts to find non-white patches

    Returns:
    list of numpy arrays containing the extracted patches
    """
    slide = openslide.OpenSlide(slide_path)
    slide_width, slide_height = slide.dimensions
    patches = []
    attempts = 0

    while len(patches) < num_patches and attempts < max_attempts:
        x = np.random.randint(0, slide_width - patch_size[0])
        y = np.random.randint(0, slide_height - patch_size[1])

        patch = slide.read_region((x, y), 0, patch_size).convert("RGB")
        patch_array = np.array(patch)

        if is_useful_patch(patch_array):
            patches.append(patch_array)

        attempts += 1

    if len(patches) < num_patches:
        print(f"Warning: Only able to extract {len(patches)} useful patches after {max_attempts} attempts.")

    return patches

def main():
    input_svs_path = '/home/jupyter-jishnu_2325_sair/Project Phase 1/Data/extracted_data/DLBCL-Morphology_13952_13952_0.svs'
    patch_size = (1388, 1040)
    num_patches = 145
```

```

train_dir          =          '/home/jupyter-jishnu_2325_sair/Project          Phase
1/Data/extracted_data/4L/Training/DLBCM'
test_dir           =          '/home/jupyter-jishnu_2325_sair/Project          Phase
1/Data/extracted_data/4L/Testing/DLBCM'
os.makedirs(train_dir, exist_ok=True)
os.makedirs(test_dir, exist_ok=True)

# Extract patches
patches = extract_patches(input_svs_path, patch_size, num_patches)

# Ensure we have exactly 145 patches
patches = patches[:num_patches]

# Split patches into training and testing sets
train_patches, test_patches = train_test_split(patches, test_size=25, train_size=120,
random_state=42)

# Save training patches
for idx, patch in enumerate(train_patches):
    output_path = os.path.join(train_dir, f'train_patch_{idx + 1}.tif')
    cv2.imwrite(output_path, cv2.cvtColor(patch, cv2.COLOR_RGB2BGR))

# Save testing patches
for idx, patch in enumerate(test_patches):
    output_path = os.path.join(test_dir, f'test_patch_{idx + 1}.tif')
    cv2.imwrite(output_path, cv2.cvtColor(patch, cv2.COLOR_RGB2BGR))

# Summary
print(f'Total patches extracted: {len(patches)}')
print(f'Training patches: {len(train_patches)}')
print(f'Testing patches: {len(test_patches)}')

if __name__ == "__main__":
    main()

#preprocessing on histological image

def is_useful_patch(image, white_threshold=220, tissue_percentage=0.5):
    """
    Check if the patch contains enough tissue (non-white area).

    Args:
    image: numpy array of the image patch
    white_threshold: pixel values above this are considered white
    tissue_percentage: minimum required percentage of non-white pixels

    Returns:
    bool: True if the patch is useful (contains enough tissue), False otherwise
    """
    # Convert to grayscale if it's a color image

```

```
if len(image.shape) == 3:
    gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)
else:
    gray = image

# Create a binary mask of non-white pixels
non_white_mask = gray < white_threshold

# Calculate the percentage of non-white pixels
non_white_percentage = np.sum(non_white_mask) / non_white_mask.size

return non_white_percentage > tissue_percentage

def extract_patches(slide_path, patch_size, num_patches=145, max_attempts=10000):
    """
    Extract patches from the slide, excluding mostly white patches.

    Args:
    slide_path: path to the .svs file
    patch_size: tuple of (width, height) for the patches
    num_patches: number of patches to extract
    max_attempts: maximum number of attempts to find non-white patches

    Returns:
    list of numpy arrays containing the extracted patches
    """
    slide = openslide.OpenSlide(slide_path)
    slide_width, slide_height = slide.dimensions
    patches = []
    attempts = 0

    while len(patches) < num_patches and attempts < max_attempts:
        x = np.random.randint(0, slide_width - patch_size[0])
        y = np.random.randint(0, slide_height - patch_size[1])

        patch = slide.read_region((x, y), 0, patch_size).convert("RGB")
        patch_array = np.array(patch)

        if is_useful_patch(patch_array):
            patches.append(patch_array)

        attempts += 1

    if len(patches) < num_patches:
        print(f"Warning: Only able to extract {len(patches)} useful patches after {max_attempts} attempts.")

    return patches

def main():
    input_svs_path = '/home/jupyter-jishnu_2325_sair/Project Phase
```

```
1/Data/extracted_data/DLBCL-Morphology_13952_13952_0.svs'
    patch_size = (1388, 1040)
    num_patches = 145

    train_dir = '/home/jupyter-jishnu_2325_sair/Project Phase 1/Data/extracted_data/4L/Training/DLBCM'
    test_dir = '/home/jupyter-jishnu_2325_sair/Project Phase 1/Data/extracted_data/4L/Testing/DLBCM'
    os.makedirs(train_dir, exist_ok=True)
    os.makedirs(test_dir, exist_ok=True)

    # Extract patches
    patches = extract_patches(input_svs_path, patch_size, num_patches)

    # Ensure we have exactly 145 patches
    patches = patches[:num_patches]

    # Split patches into training and testing sets
    train_patches, test_patches = train_test_split(patches, test_size=25, train_size=120,
    random_state=42)

    # Save training patches
    for idx, patch in enumerate(train_patches):
        output_path = os.path.join(train_dir, f'train_patch_{idx + 1}.tif')
        cv2.imwrite(output_path, cv2.cvtColor(patch, cv2.COLOR_RGB2BGR))

    # Save testing patches
    for idx, patch in enumerate(test_patches):
        output_path = os.path.join(test_dir, f'test_patch_{idx + 1}.tif')
        cv2.imwrite(output_path, cv2.cvtColor(patch, cv2.COLOR_RGB2BGR))

    # Summary
    print(f'Total patches extracted: {len(patches)}')
    print(f'Training patches: {len(train_patches)}')
    print(f'Testing patches: {len(test_patches)}')

if __name__ == "__main__":
    main()

#data preprocessing

IMG_SIZE = (224, 224)
ZOOM_RANGE = 0.1
SCALE_FACTOR = 1.0 / 255.0

input_dir = "/home/jupyter-jishnu_2325_sair/Project Phase 1/Data/my_folder/"
output_dir = "/home/jupyter-jishnu_2325_sair/Project Phase 1/Data/Preprocessed"

datagen = ImageDataGenerator(
    rescale=SCALE_FACTOR,
```

```
        zoom_range=ZOOM_RANGE
    )

def preprocess_images(input_dir, output_dir):
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    for root, dirs, files in os.walk(input_dir):
        for file_name in files:
            if file_name.lower().endswith('.tif'):
                input_path = os.path.join(root, file_name)
                relative_path = os.path.relpath(root, input_dir)
                output_subdir = os.path.join(output_dir, relative_path)

                if not os.path.exists(output_subdir):
                    os.makedirs(output_subdir)

                output_path = os.path.join(output_subdir, file_name)

                try:
                    # Open and resize the image
                    img = Image.open(input_path)
                    img = img.resize(IMG_SIZE)

                    # Convert to numpy array and preprocess
                    img_array = np.array(img)
                    img_array = img_array.astype(np.float32) * SCALE_FACTOR
                    img_array = datagen.random_transform(img_array)

                    # Convert back to uint8 and save
                    img_processed = Image.fromarray((img_array * 255).astype(np.uint8))
                    img_processed.save(output_path)

                except Exception as e:
                    print(f"Error processing {input_path}: {e}")

# Process both Training and Testing directories
sub_dirs = ["Training", "Testing"]
for sub_dir in sub_dirs:
    preprocess_images(
        input_dir=os.path.join(input_dir, sub_dir),
        output_dir=os.path.join(output_dir, sub_dir)
    )

print(f"Preprocessing completed. Preprocessed images saved to {output_dir}")
```